PROJECT DESIGN PHASE & PROPOSAL

Comprehensive Proposed Solution & System Architecture

Doc ID: PRJ-PROP-2026-V3
Date: October 2026
Status: Under Review

1. EXECUTIVE SUMMARY

This Project Proposal defines the comprehensive technical architecture, design specifications, and implementation strategy for the Next-Gen Enterprise Resource Planning (ERP) Integration Platform. The system aims to bridge legacy data stores with modern cloud microservices, eliminating cross-departmental data silos, reducing automated sync latency from 24 hours to sub-seconds, and providing operational teams with real-time actionable analytics.

99.95%	< 250ms	3.5x	40%
TARGET UPTIME SLI	API LATENCY (P95)	THROUGHPUT MULTIPLIER	OPEX REDUCTION

Problem Statement

Current operational workflows suffer from fractured legacy systems, causing severe data sync delays, manual reconciliation overheads exceeding 120 hours/month, and recurring inventory stockouts during peak transactional periods.

Proposed Solution Overview

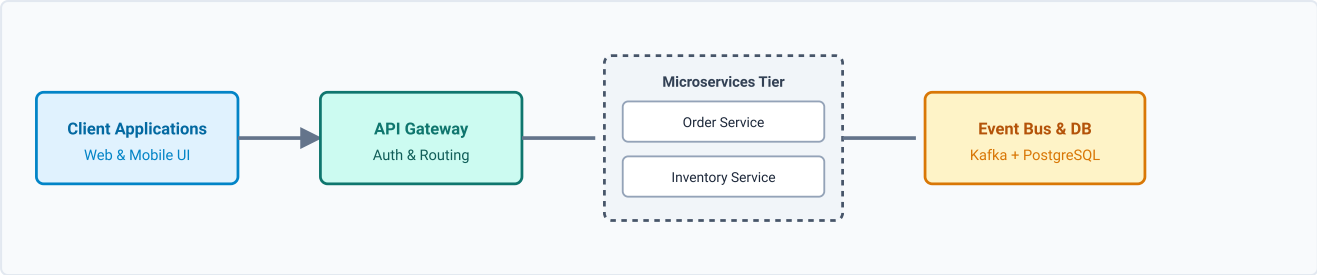
An event-driven microservices architecture hosted on modern cloud infrastructure, leveraging distributed event streaming (Apache Kafka) and containerized API gateways to achieve seamless data flow and real-time processing.

2. STRATEGIC PROJECT OBJECTIVES

Objective ID	Key Performance Indicator	Baseline	Target Goal
OBJ-01	Real-time inventory catalog synchronization	24-hour batch sync	< 500 ms stream
OBJ-02	Order processing automation throughput	450 orders / hr	2,500 orders / hr
OBJ-03	System downtime / maintenance loss	14.2 hrs / month	< 0.5 hrs / month
OBJ-04	Customer query resolution lead time	4.5 business hours	15 minutes

3. PROPOSED SOLUTION ARCHITECTURE

The proposed architecture is structured around a multi-tier decoupling strategy. The presentation layer utilizes modular micro-frontends, backed by a unified GraphQL gateway. Core domain capabilities are implemented as stateless containerized microservices communicating via an event broker.



4. FUNCTIONAL MODULES & TECHNICAL STACK

System Module	Core Responsibilities	Selected Tech Stack
API Gateway Layer	Request routing, rate limiting, JWT authentication, schema stitching	Kong API Gateway, Redis, Node.js
Core Services Engine	Business logic processing, state mutation, domain event dispatching	Go (Golang), Docker, Kubernetes
Event Streaming Platform	Asynchronous message delivery, audit logging, system-wide event broadcasting	Apache Kafka, Strimzi Operator
Persistence & Cache	ACID transactional stores, relational schemas, high-speed key-value caches	PostgreSQL 16, Redis Enterprise

> **Security & Compliance Framework**
> All intra-service communication will be encrypted using Mutual TLS (mTLS) enforced via an Istio Service Mesh. End-to-end data encryption at rest (AES-256) and in transit (TLS 1.3) will be applied alongside OAuth 2.0 / OIDC role-based access controls to achieve SOC 2 Type II and GDPR compliance.

5. PROJECT EXECUTION METHODOLOGY

The engagement will follow an Agile Scrum framework with bi-weekly sprint cadences, ensuring rapid feedback loops, iterative component releases, and high alignment with business stakeholders throughout the 24-week engineering lifecycle.

6. WORK BREAKDOWN & IMPLEMENTATION PHASES

Phase	Phase Name	Key Deliverables	Duration
Phase 1	Discovery & Blueprinting	Architecture specification, API contracts, infrastructure IaC scripts	Weeks 1–4
Phase 2	Core Platform Engineering	API Gateway setup, Event bus deployment, Auth integration	Weeks 5–10
Phase 3	Services & Data Migration	Microservices coding, legacy database connectors, pipeline ETL	Weeks 11–18
Phase 4	UAT & Performance Tuning	Load testing (k6), security penetration audit, user training	Weeks 19–22
Phase 5	Production Cutover	Blue-green deployment, live monitoring, post-launch hypercare	Weeks 23–24

Quality Assurance Strategy

Automated CI/CD testing pipelines targeting 85%+ unit coverage, integrated regression suites, end-to-end Cypress UI testing, and continuous static analysis via SonarQube.

Governance & Reporting

Weekly sprint reviews, bi-weekly steering committee updates, real-time Jira dashboard access for stakeholders, and structured change management protocols.

7. RISK ASSESSMENT & MITIGATION STRATEGY

Risk Area	Impact	Likelihood	Mitigation Plan
Legacy API Bottlenecks	High	Medium	Implement change-data-capture (CDC) triggers directly on underlying databases to bypass slow legacy HTTP interfaces.
Data Format Mismatches	Medium	High	Enforce strictly validated JSON Schema contracts at the API gateway layer prior to payload ingestion.
Scope Creep in Phase 3	High	Low	Establish a formal Change Request Board (CRB); defer non-critical feature enhancements to Phase 2 backlog.

8. PROJECT BUDGET & RESOURCE ALLOCATION

Cost Center / Resource Category	Allocation Details	Estimated Cost (USD)
Engineering Personnel	1 Architect, 4 Senior Devs, 2 QA, 1 DevOps (24 wks)	$240,000
Cloud Infrastructure & Tooling	AWS staging/prod environments, Kafka SaaS, CI licenses	$35,000
Security & Audit Services	Third-party penetration testing and compliance audit	$18,000
Contingency Reserve	10% risk contingency buffer	$29,000
	Total Proposed Budget:	**$322,000**

Formal Approval & Authorization

By signing below, the stakeholders endorse the proposed project architecture, scope, timeline, and budget estimation contained within this document.

_____

Sponsor / Lead Stakeholder
Date: _____

_____

Lead Systems Architect
Date: _____